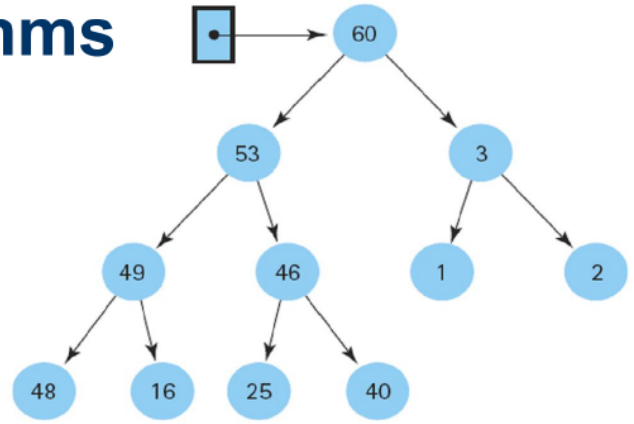
Data Structures & Algorithms



CHAPTER 0: INTRODUTION

What this course is about ?

- conceptual and concrete ways to organize data for efficient storage and efficient manipulation
- Employment of this data structures in the design of efficient algorithms

Data Structure Example Applications

1. How does Google quickly find web pages that contain a search term?
2. What's the fastest way to broadcast a message to a network of computers?
3. How does your operating system track which memory (disk or RAM) is free?

What is a Data Structure Anyway?

- **It's an agreement about:**

1. How to store a collection of objects in memory,
2. What operations we can perform on that data,
3. The algorithms for those operations, and
4. How time and space efficient those algorithms are.

Ex. vector in C++:

1. Stores objects sequentially in memory
2. Can access, change, insert or delete objects
3. Algorithms for insert & delete will shift items as needed
4. Space: O(n), Access/change = O(1), Insert/delete = O(n)

Why do we need them ?

- Computers take on more and more complex tasks
  - Imagine: index of 8 billion pages ! (Google)
- Software implementation and maintenance is difficult.
- Clean conceptual framework allows for more efficient and more correct code

Why do we need them

- Requirements for a good software:
  - Clean Design
  - Easy maintenance
  - Reliable (no core dumps)
  - Easy to use
  - Fast algorithms

→**Efficient data structures**

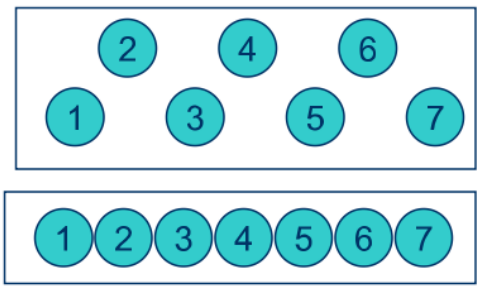
→**Efficient algorithms**

Basic Data Structures

- Structures include
  - linked lists
  - Stack, Queue
  - binary trees
  - …and others

What is Algorithm?

- **Algorithm:**
  - A computable set of steps to achieve a desired result
  - Ralationship to Data Structure
    - Example: Find an element



Sumary

Algorithms + Data Structures = Programs

Algorithms ↔ Data Structures

ADT

- ADT = Abstract Data Types
- ADT is A logical view of the data objects together with specifications of the operations required to create and manipulate these objects.

Abstract Data Types (ADT)

- Data storage & operations encapsulated by an ADT.
- ADT specifies permitted **operations** as well as **time** and **space** guarantees.
- User unconcerned with how it's implemented.

Usual Data Types

- Example :Integer, Double, Boolean
- You do not know the implementation of these.
- There are some operations which can be done with these (add, multiply, read, write)
- Programmer just uses these operations without knowing their implementation.

Abstract Data Types

- Implement these data types once in the program
- Describe these operations once
- Use these operations again and again without going into the implementation
- However, the complexity of these operation depend on the implementation and should not be considered constant
- An ADT may use a different ADT

What is the different between ADT and data structure ?

Chapter 1: C LANGUAGE

1. *ADDRESS*
2. *POINTERS*
3. *ARRAYS*
4. *PASS ARRAY TO FUNCTION*
5. *ADDRESS OF EACH ELEMENT IN AN ARRAY*
6. *ACCESSING & MANIPULATING AN ARRAY USING POINTERS*
7. *TWO-DIMENSIONAL ARRAY*
8. *POINTER ARRAYS*
9. *STRUCTURES*
10. *STRUCTURE POINTERS*

1. ADDRESS

For every variable there are two attributes:
<u>address</u> and <u>value</u>

In memory with address 3: value: 45.
In memory with address 2: value "Dave"

1	4096
2	"Dave"
3	45
4	"Matt"
5	95.5
6	"wbru"
7	0
8	"zero"

```
cout << "Value of 'y' is: " << y << "\n";
cout << "Address of 'y' is: " << &y << "\n\n";
```

2. POINTERS

1. ***is a variable whose value is also an address.***
2. ***A pointer to an integer is a variable that can store the address of that integer***

ia: value of variable

&ia: address of ia

***ia means you are printing the value at the location specified by ia**

1000, i	10
4000, ia	—, 1000

2. POINTERS (cont)

```
int i;        //A
int * ia;     //B
  cout<<“The address of i is “<<&i<<endl ;
  cout<<"The address of ia is “<< &ia;

 i = 10;       //C
 ia = &i;   //D

  cout<<"after assigning value:\n";
  cout<<"The address of i is value= “<<&i<<endl;
  cout<<"The address of ia is “”<<&ia<<“ value= “<< ia<<“ point to“<<*ia;
```

2. POINTERS (cont)

Points to Remember

- **Pointers give a facility to access the value of a variable indirectly.**
- **You can define a pointer by including a * before the name of the variable.**
- During the lifetime of the variable, the address is not changed but the value may change.
- **You can get the address where a variable is stored by using &.**

3. ARRAYS

1. *An array is a data structure*
2. *used to process multiple elements with the same data type when a number of such elements are known.*
3. *An array is a composite data structure; that means it had to be constructed from basic data types such as array integers.*

1. *int a[5];*
2. *for(int i = 0;i<5;i++)*
   1. *{a[i]=i; }*

4. PASS ARRAY TO FUNCTION

نستطيع تمرير المصفوفة كمعامل في الدالة

```
#include<iostream>
using namespace std;
void printarray (int a[])
{
    for (int i=0;i<5;i++)
        cout<<"value in array in location"<<i<<"="<<a[i]<<endl;

}
void main(){
    int arr[5];
    for(int j=0;j<5;j++)
        arr[j]=2*j;
    cout<<"praint an array element \n";
    printarray(arr);
}
```

5. ADDRESS OF EACH ELEMENT IN AN ARRAY .

Each element of the array has a memory address.

عناوين عناصر المصفوفة

```
#include<iostream>
using namespace std;
void printdetail (int a[])
{
    for (int i=0;i<5;i++)
        cout<<"value in array "<<a[i]<<"at address "<<&a[i]<<endl;

}
void main(){
    int arr[5];
    for(int j=0;j<5;j++)
        arr[j]=2*j;
    cout<<"print an array element with address\n";
    printarray(arr);
```

6. ACCESSING & MANIPULATING AN ARRAY USING POINTERS

- *You can access an array element by using a pointer.*
- *If an array stores integers->use a pointer to integer to access array elements.*

يمكن الوصول الى عناصر المصفوفة باستخدام المؤشرات

ملاحظة مهمة / اسم المصفوفة هو عبارة عن مؤشر ثابت يشير الى اول عنصر فى المصفوفة

```
#include<iostream>
using namespace std;
void printarray_usingpointer (int a[])
{
    int *p;
    p=a;
    for (int i=0;i<5;i++){
        cout<<"value in array in location "<<i<<"is"<<*p<<endl;
        p++;
    }
}
void main(){
    int arr[5];
    for(int j=0;j<5;j++)
        arr[j]=2*j;
    cout<<"print an array element using pointer\n";
    printarray_usingpointer(arr);
}
```

```
int a[5];    int *p;
a=p; //error
p=a; //OK
```

7. TWO-DIMENSIONAL ARRAY

المصفوفة ذات البعدين .

int a[3][2];

```
print an array element
0       1
1       3
2       5
```

```
#include<iostream>
using namespace std;
int main(){
    int arr[3][2];
    for (int i=0;i<3;i++)
        for(int j=0;j<2;j++)
            arr[i][j]=i+j+i*j;
    cout<<"print an array element\n";
    int *p;
    p=&arr[0][0];
    for(int i=0;i<3;i++)
    {
        for(int j=0;j<2;j++)
        {
            cout<<*p<<"\t";
            p++;
        }
        cout<<endl;
    }
    return 0;
}
```

8. POINTER ARRAYS

- *You can define a pointer array (similarly to an array of integers).*
- *In the pointer array, the array elements store the pointer that points to integer values.*

مصفوفة المؤشرات :

هي عبارة عن مصفوفة عناصرها عبارة عن مؤشرات لقيم في الذاكرة

أي ان عناصر المصفوفة تحتوي عناوين .

8. POINTER ARRAYS (cont)

```
#include<iostream>
using namespace std;
void main(){
    int x=1,y=2,z=3;
    int *a[3];
    a[0]=&x,a[1]=&y,a[2]=&z;
    cout<<"&x="<<&x<<" &y="<<&y<<" &z"<<&z<<endl;
    for(int i=0;i<3;i++)
        cout<<i<<"element is = "<<*a[i]<<"and address= "<<a[i]<<endl;
}
```

```
C:\Windows\system32\cmd.exe
&x=0036F840 &y=0036F834 &z=0036F828
0 elemnet is=1 and address=0036F840
1 elemnet is=2 and address=0036F834
2 elemnet is=3 and address=0036F828
Press any key to continue . . .
```

9. STRUCTURES

- *Structures are used when you want to process data of multiple data types*
- *But you still want to refer to the data as a single entity*
- *Access data: structureveriable.membername*

9. STRUCTURES (cont)

```
#include<iostream>
using namespace std;
struct students{
    char name[20];
    float GPA;
};
void main(){
    students st_1,st_2,st_3;
    cout<<"Enter the information of students 1\nname:";
        cin>>st_1.name;
    cout<<"GPA: ";
    cin>>st_1.GPA;
    cout<<"information of students 1"<<endl;
    cout<<st_1.name<<"\t"<<st_1.GPA;
}
```

students st_1

```
C:\Windows\system32\cmd.exe
Enter the information of students 1
name: maha
GPA: 3.88
information of students 1
name: maha        GPA: 3.880000
Press any key to continue . . .
```

10. STRUCTURE POINTERS

Process the structure using a structure pointer

```
#include<iostream>
using namespace std;
struct rectangle{
    int length;
    int width;
};
void main(){
    rectangle rect1;
    cout<<"Enter rectangle lentgh:\n";
    cin>>rect1.length;
    cout<<"Enter rectangle witdht:\n";
    cin>>rect1.width;

    cout<<"rectangle information\nlength:"<<rect1.length<<"\twidth:"<<rect1.width<<endl;

    rectangle *rect2;
    rect2=&rect1;

    rect2->length=20;
    cout<<"\nrectangle information\nlength:"<<rect1.length<<"\twidth:"<<rect1.width<<endl; }
```

```
C:\Windows\system32\cmd.exe
Enter rectangle lentgh:
60
Enter rectangle witdht:
88
rectangle information
length:60        width:88

rectangle information
length:20        width:88
Press any key to continue . . .
```

End

ملاحظة : المحاضرة القادمة الكويز (١)